SWING SENSE

Plano de Respostas aos Riscos de Segurança da Informação

Entrega 2 — Relatório de Mitigações Implementadas no Código — Redes de Computadores e Cibersegurança

Projeto: Swing Sense — rede social de treinos de tênis com raquete inteligente (ESP32)
Data de criação/atualização: 18/10/2026
Versão: 2.0 (evolução do Plano de Gerenciamento de Riscos v1.0 — Entrega 1)
Componentes tratados: Backend (Dart/shelf), App (Flutter), Banco (PostgreSQL/Supabase), Container (Docker)
Integrantes do grupo:
Antônio Petri • Gabriel Higobassi Paschoal
Lucas Silva Maciel • Thiago Akira Higa Mitami

Entregáveis: este relatório (HTML/PDF) e o código-fonte em `SRC/Swing Sense/` com as correções e a suíte de testes `swing_sense_backend/test/security_test.dart`.

Plano de Respostas aos Riscos

Projeto: Swing Sense | Data: 18/10/2026 | Versão: 2.0

1. Introdução

Este documento detalha as estratégias e ações específicas aplicadas para lidar com os riscos (vulnerabilidades) identificados no projeto Swing Sense. Ele dá continuidade ao *Plano de Gerenciamento de Riscos* da Entrega 1, que encontrou e provou 12 vulnerabilidades (R-01 a R-12), e passa da **identificação** para a **resposta**: cada risco recebeu uma estratégia (prevenir, transferir, mitigar ou aceitar), ações técnicas concretas **já implementadas no código** e uma verificação.

Nesta entrega também foi feita uma nova rodada de revisão de código (SAST) em todo o backend e no app, que encontrou **6 riscos adicionais (R-13 a R-18)**. O objetivo acadêmico é demonstrar o ciclo completo: *encontrar → documentar → corrigir → comprovar*, sem alterar o contrato da API nem a usabilidade do app.

> **Resumo executivo.** Dos 18 riscos, **14** estão **Concluídos** (correção no código + teste automatizado), **3** estão **Em Andamento** (código pronto, dependem de implantação/processo: R-04, R-09, R-12) e **1** está **Não Iniciado** (R-18, com justificativa). Os dois riscos **Extremos** (R-01, R-02) foram eliminados. Nível antes → depois: Extremo 2→0; Alto 4→0; Médio 8→5; Baixo 4→11; Muito Baixo 0→2.

2. Tabela de Respostas aos Riscos

Núcleo do plano: a resposta para cada risco priorizado. "Nível" é o nível inicial (matriz da Entrega 1); o nível residual após as ações está na Seção 5. Responsáveis são papéis definidos na Entrega 1 (Coordenação de Segurança: Lucas Silva Maciel; Equipe de Desenvolvimento: demais integrantes). Prazos marcados como *proposto* ainda não ocorreram.

ID do Risco	Descrição do Risco	Nível (Prioridade)	Estratégia de Resposta	Ações Específicas Planejadas / Aplicadas	Responsável pela Ação	Prazo	Status
R-01	Segredo JWT padrão/hardcoded no código	**Extremo**	Prevenir (Evitar)	1. Removido o valor padrão: `SecurityConfig.fromEnv()` (`utils/security_config.dart`) exige segredo ≥ 32 caracteres e que não seja nenhum dos exemplos públicos. 2. Com `APP_ENV=production` a API **recusa subir** com segredo ausente/fraco (falha antes de abrir o banco). 3. Em desenvolvimento, segredo vazio/fraco gera um segredo **aleatório temporário** (48 bytes, `Random.secure`) com aviso — o "plug and play" local continua e nenhuma chave conhecida é aceita. 4. `.env.example` sem segredo reutilizável, com instrução `openssl rand -base64 48`. Tokens agora levam `iss` e `jti` e a verificação exige o emissor.	Coordenação de Segurança	18/10/2026	Concluído
R-02	IDOR na ingestão de telemetria (POST /sessions/:id/events)	**Extremo**	Prevenir (Evitar)	1. `session_routes.dart`: a rota carrega a sessão e exige `session.user_id == request.userId` (mesmo padrão do `GET /:id/events`); caso contrário **404** (não revela a existência do ID). 2. IDs que não são UUID retornam 404 antes de chegar ao banco.	Equipe de Desenvolvimento	18/10/2026	Concluído
R-03	IDOR em curtidas e comentários (/feed/:id/like, /comments)	**Médio**	Mitigar	1. `feed_routes.dart`: função `canInteract` — a sessão precisa ser do próprio usuário, ou **concluída e de alguém que ele segue** (a mesma regra do	Equipe de Desenvolvimento	18/10/2026	Concluído

				feed, então a experiência do app não muda). 2. Aplicada em curtir, listar e criar comentários; comentário limitado a 500 caracteres.			
R-04	Enumeração de usuários no cadastro	**Médio**	Mitigar + Aceitar (Ativo)	1. Limite de **10 cadastros/hora por IP** (`AuthLimiters.registerPerIp`) → HTTP 429 com `Retry-After`: impede varrer listas de e-mails em escala. 2. Login: bcrypt "descartável" quando o e-mail não existe, igualando o tempo de resposta (ver R-15). 3. **Decisão consciente:** o 409 foi mantido porque o app o usa para orientar o usuário; trocá-lo exige fluxo de confirmação por e-mail (não existe ainda). Risco residual aceito de forma ativa (rate limit + monitoramento) até a implantação do e-mail de confirmação.	Equipe de Desenvolvimento	15/11/2026 (proposto)	Em Andamento
R-05	Ausência de proteção contra força bruta (login)	**Alto**	Mitigar	1. `utils/rate_limiter.dart`: limitador de janela deslizante (15 min) em três camadas — 30 tentativas/IP, **5 falhas por IP+e-mail**, 20 falhas por e-mail (qualquer IP). 2. Bloqueio responde **429 + Retry-After**; login correto zera o contador do par. O par IP+e-mail evita que um atacante tranque a conta da vítima a partir de outro IP. 3. IP real só é lido de `X-Forwarded-For` quando `TRUST_PROXY=true` (senão o atacante forjaria o header).	Equipe de Desenvolvimento	18/10/2026	Concluído
R-06	Política de senha fraca	**Médio**	Mitigar	1. `utils/validators.dart` › `validatePassword`: 8–128 caracteres, ao menos uma letra e um número, fora de lista de senhas comuns, sem conter o e-mail/ser igual ao nome. O teto de 128 evita DoS no bcrypt. 2. App: validador da tela de cadastro alinhado (`signup_screen.dart`). 3. **Compatibilidade:** a política vale só no *cadastro*; o login continua aceitando senhas antigas de 6+ caracteres, então nenhuma conta existente é bloqueada.	Equipe de Desenvolvimento	18/10/2026	Concluído
R-07	CORS permissivo (Access-Control-Allow-Origin: *)	**Baixo**	Mitigar	1. `middleware/security_middleware.dart` › `corsAllowlist`: allowlist por `CORS_ALLOWED_ORIGINS`; origem permitida é refletida com `Vary: Origin`; preflight de origem não permitida → 403. 2. Apps nativos (Android/iOS) não enviam `Origin` e não são afetados. Fora de produção `localhost` segue liberado (`flutter run -d chrome` continua funcionando).	Equipe de Desenvolvimento	18/10/2026	Concluído
R-08	Tokens JWT de longa duração sem revogação	**Médio**	Mitigar	1. Validade padrão **24 h**, teto de 72 h (`JWT_EXPIRES_IN_HOURS` acima disso é reduzido com aviso). 2. Cada token ganha um `jti`; novo `POST /auth/logout` o inclui numa lista de revogação em memória (limpa após o `exp`). O app chama o logout em modo "melhor esforço" (offline não impede sair). 3. Tokens emitidos antes desta versão (sem `jti`) deixam de valer: o app trata o 401 no `bootstrap()` e leva o usuário ao login	Equipe de Desenvolvimento	18/10/2026	Concluído

				— **um novo login único** após a atualização.			
R-09	Transporte sem TLS (HTTP em texto claro)	**Alto**	Mitigar (+ Transferir TLS ao proxy/provedor)	1. **API:** TLS nativo opcional (`TLS_CERT_PATH` / `TLS_KEY_PATH`) ou atrás de proxy com `TRUST_PROXY=true`; **HSTS** em produção/HTTPS; aviso na subida se produção rodar em HTTP puro; demais cabeçalhos (`nosniff`, `X-Frame-Options`, `CSP`, `Cache-Control: no-store`, `Referrer-Policy`). 2. **Android:** `usesCleartextTraffic="false"` no manifesto principal (release); debug/profile mantêm HTTP para a API local — o desenvolvimento não muda. 3. **App:** em build de *release*, `ApiClient` bloqueia URL `http://` que não seja localhost/emulador; basta compilar com `--dart-define=API_BASE_URL=https://…`. 4. **Pendente (implantação):** publicar a API com certificado válido (proxy/plataforma de nuvem). O código está pronto; o certificado depende do ambiente de hospedagem.	Coordenação de Segurança	15/11/2026 (proposto)	Em Andamento
R-10	Container executando como root em modo de desenvolvimento	**Baixo**	Mitigar	1. `Dockerfile` multi-estágio: `dart compile exe` (AOT) e imagem final `FROM scratch` (sem shell/gerenciador de pacotes), `USER 10001`, `HEALTHCHECK` pelo próprio binário (`--healthcheck`). 2. `docker-compose.yml`: `read_only`, `cap_drop: ALL`, `no-new-privileges`, limites de memória e processos.	Equipe de Desenvolvimento	18/10/2026	Concluído
R-11	Falta de validação/limite de payload (abuso de armazenamento/DoS)	**Baixo**	Mitigar	1. `bodyLimit`: corpo máximo de 256 KiB (`MAX_BODY_BYTES`), checado por `Content-Length` e em streaming (cobre *chunked*) → 413. 2. Máx. 500 eventos por requisição (o app envia lotes pequenos a cada 6 s); `eventType` restrito a `[a-z][a-z0-9_]{0,39}` (aceita `shot/ace/fault` e os tipos futuros do ESP32); faixas: velocidade 0–400 km/h, rotação 0–10000 rpm, ângulo ±360°. 3. Limites de texto: comentário 500, notas 2000, bio 300, título 120 etc.; números das sessões/metas com faixa (evita *overflow* de `NUMERIC(5,1)` → 500).	Equipe de Desenvolvimento	18/10/2026	Concluído
R-12	Exposição de credenciais do banco (Supabase) por vazamento de segredo	**Alto**	Transferir + Mitigar	1. `.dockerignore` exclui `.env*` da imagem (o `.env` chega só como variável de ambiente via compose). 2. Workflow `.github/workflows/security.yml`: **gitleaks** em cada push/PR + `dart analyze` e `dart test`. 3. Auditoria do histórico git: nenhum `.env` real ou senha foi versionado (só os placeholders do `.env.example`). 4. **Transferir:** usar Supabase Vault / GitHub Actions Secrets para produção. **Pendente:** rotação periódica e papel de banco com menor privilégio.	Coordenação de Segurança	15/11/2026 (proposto)	Em Andamento
R-13 (novo)	Exposição de dados pessoais no perfil	**Alto**	Prevenir (Evitar)	1. Novo mapper `userPublicToJson` (`mappers.dart`): nome, avatar, bio, nível, cidade e data de cadastro. E-	Equipe de Desenvolvimento	18/10/2026	Concluído

	público (GET /users/:id)			mail/nascimento/role só voltam em `/users/me` e `PUT /users/me`.			
R-14 (novo)	Sem tratamento global de erros (HTTP 500 / vazamento) e IDs sem validação	**Médio**	Mitigar	1. `errorHandler`: JSON inválido/tipos errados → 400; erros de dado do Postgres (22P02, 22003, 22001, 23503…) → 400 "Dados invalidos"; qualquer outro → 500 genérico "Erro interno" (detalhes só no log do servidor). 2. IDs de rota validados com `isUuid` antes de consultar o banco (404 para formato inválido). Resposta de erro nunca inclui stack trace/SQL.	Equipe de Desenvolvimento	18/10/2026	Concluído
R-15 (novo)	Diferença de tempo no login revela se o e-mail existe (timing)	**Médio**	Mitigar	1. Um hash bcrypt descartável é verificado também quando o usuário não existe, equalizando o custo.	Equipe de Desenvolvimento	18/10/2026	Concluído
R-16 (novo)	Validação insuficiente de entrada e integridade dos dados	**Médio**	Mitigar	1. Validadores centrais (`utils/validators.dart`): `validateText/Number/HttpUrl/Date`, allowlists de `level` e `status` da meta, cor hexadecimal, `isSlug` para tipos. 2. `POST /sessions`: `trainingId` deve existir e `deviceId` deve pertencer ao usuário autenticado. `POST /users/:id/follow` valida existência do alvo. 3. **Compatibilidade:** os valores que o app envia hoje (`iniciante…profissional`, `active/completed`, tipos de meta e eventos `shot/ace/fault`) foram conferidos no código do app e continuam aceitos.	Equipe de Desenvolvimento	18/10/2026	Concluído
R-17 (novo)	Curingas de ILIKE sem escape na busca de usuários	**Baixo**	Prevenir (Evitar)	1. `escapeLike` escapa `% _ \` e a consulta usa `ILIKE … ESCAPE '\'`; consulta limitada a 50 caracteres (e 20 resultados, já existente).	Equipe de Desenvolvimento	18/10/2026	Concluído
R-18 (novo)	Token JWT guardado sem criptografia no dispositivo (SharedPreferences)	**Médio**	Mitigar (planejado)	1. Recomendado: migrar para `flutter_secure_storage` (Keystore/Keychain). **Não aplicado nesta entrega** porque exige nova dependência, ajuste nativo (minSdk 23, Keychain) e testes em dispositivo que não foi possível executar aqui; trocar às cegas arriscaria quebrar o login do app. 2. Mitigação parcial já ativa: validade de 24 h e revogação no logout (R-08).	Equipe de Desenvolvimento	29/11/2026 (proposto)	Não Iniciado

3. Detalhamento das Estratégias de Resposta

3.1. Estratégias para Ameaças (Riscos Negativos) e como foram usadas

- **Prevenir (Evitar):** mudar o projeto para eliminar a ameaça. *Aplicado em:* R-01 (o segredo padrão deixou de existir), R-02 (a rota passou a exigir posse), R-13 (e-mail/nascimento de terceiros nem saem do servidor) e R-17 (curingas escapados).
- **Transferir:** repassar o impacto/resposta a um terceiro. *Aplicado em:* R-12 (guarda de segredos em cofre do provedor: Supabase Vault / GitHub Secrets) e parte de R-09 (terminação TLS delegada a proxy/plataforma de nuvem). Não elimina o risco, apenas muda quem o gerencia.
- **Mitigar:** reduzir probabilidade e/ou impacto a um nível aceitável. *Aplicado em:* R-03, R-05, R-06, R-07, R-08, R-09, R-10, R-11, R-14, R-15, R-16 e R-18 (rate limit, política de senha, allowlist de CORS, tokens curtos com revogação, hardening de container, validação de entrada).
- **Aceitar:** *Ativa* — R-04: o 409 do cadastro foi mantido por usabilidade e compensado com rate limit por IP e monitoramento; *Passiva* — risco residual de revogação de token apenas em memória (R-08), de baixa prioridade dada a validade de 24 h.

3.2. Estratégias para Oportunidades (Riscos Positivos)

Nenhuma oportunidade foi registrada neste ciclo. Como ganho colateral (**Explorar**), a suíte `security_test.dart` e o workflow de CI passam a funcionar como rede de proteção contra regressões em todas as próximas entregas, inclusive na integração com o ESP32 real.

3.3. Princípios seguidos nas correções (para não quebrar o projeto)

Preocupação	Como foi tratada
Contrato da API e rotas	Nenhuma rota removida ou renomeada; mesmos JSONs. Única rota nova: `POST /auth/logout`. Perfis de terceiros deixam de trazer campos que o app não usa.
Contas existentes	Política de senha vale só no *cadastro*; login aceita senhas antigas. Tokens antigos expiram (um novo login).
Desenvolvimento local	Sem `JWT_SECRET` em dev a API sobe com segredo temporário; CORS libera `localhost`; Android debug mantém HTTP; `docker-compose` usa `APP_ENV=development` por padrão.
Comportamento do app	Valores enviados pelo app (níveis, status, tipos de meta, eventos) foram conferidos no código do app antes de montar as allowlists; mensagens de erro continuam no campo `error` que o `ApiClient` exibe.
Regras de negócio sociais	A visibilidade para curtir/comentar replica exatamente a regra do feed (próprias + seguidos concluídos).

4. Matrizes de Probabilidade × Impacto

Definições de probabilidade e impacto: as mesmas da Seção 6 do Plano v1.0 (escala de Muito Baixa a Muito Alta; impacto medido em confidencialidade, integridade e disponibilidade).

4.1. Antes das respostas (inicial)

Prob. ↓ / Impacto →	Muito Baixo	Baixo	Médio	Alto	Muito Alto
Muito Alta	**Médio**	**Alto**	**Alto**	**Extremo**	**Extremo**
Alta	**Baixo**	**Médio** R-04, R-14	**Alto** R-05, R-13	**Alto** R-02	**Extremo** R-01

Média	**Baixo** R-17	**Médio** R-07, R-11, R-15	**Médio** R-03, R-06, R-08, R-16	**Alto** R-09	**Alto**
Baixa	**Muito Baixo**	**Baixo**	**Baixo** R-10	**Médio** R-18	**Médio** R-12
Muito Baixa	**Muito Baixo**	**Muito Baixo**	**Baixo**	**Baixo**	**Médio**

4.2. Depois das respostas (residual)

Prob. ↓ / Impacto →	Muito Baixo	Baixo	Médio	Alto	Muito Alto
Muito Alta	**Médio**	**Alto**	**Alto**	**Extremo**	**Extremo**
Alta	**Baixo**	**Médio**	**Alto**	**Alto**	**Extremo**
Média	**Baixo**	**Médio** R-04	**Médio**	**Alto**	**Alto**
Baixa	**Muito Baixo**	**Baixo** R-11, R-14, R-15, R-16	**Baixo** R-05, R-06, R-08	**Médio** R-09, R-18	**Médio** R-12
Muito Baixa	**Muito Baixo** R-17	**Muito Baixo** R-07	**Baixo** R-03, R-10, R-13	**Baixo** R-02	**Médio** R-01

Observação de consistência: no Plano v1.0 quatro riscos foram registrados com nível diferente do que a matriz calcularia: R-02 (Alta × Alto → matriz: Alto; registrado Extremo), R-12 (Baixa × Muito Alto → Médio; registrado Alto) e R-07/R-11 (Média × Baixo → Médio; registrados Baixo). Mantivemos a classificação original do registro para dar continuidade à Entrega 1 (R-02 e R-12 por critério conservador: integridade do feed e exposição de toda a base); por isso, na matriz 4.1 esses IDs aparecem na célula calculada, mas as demais tabelas usam o nível do registro. Os níveis residuais seguem a matriz.

Extremo/Alto: atenção imediata. Médio: monitorar de perto. Baixo/Muito Baixo: rotina. Os Médios residuais (R-01, R-04, R-09, R-12, R-18) têm impacto potencial alto e dependem de gestão de segredos, implantação com HTTPS e armazenamento seguro no app.

5. Detalhamento, Evidências e Verificação por Risco

Para cada risco: nível antes/depois, o que foi alterado no código, evidência e risco residual. As evidências "originais" vêm do Relatório da Entrega 1 (DAST); as "depois" vêm da suíte automatizada (Seção 6). Riscos novos foram encontrados por revisão de código (SAST).

R-01 — Segredo JWT padrão/hardcoded no código Extremo

Categoria: Técnico • **Referência:** A02:2021 – Falhas Criptográficas / A07 – Identificação e Autenticação

Antes: prob. Alta × impacto Muito Alto = Extremo → **Depois (residual):** prob. Muito Baixa × impacto Muito Alto = Médio • **Status:** Concluído

Descrição: Sem a variável `JWT_SECRET`, a API assinava tokens com um segredo fixo e público no repositório (`dev-secret-troque-em-producao`), permitindo forjar o token de qualquer usuário.

Evidência original: Entrega 1: token forjado com o segredo padrão acessou `/users/me` da vítima (HTTP 200) sem a senha.

Resposta implementada (Prevenir (Evitar)):

- Removido o valor padrão: `SecurityConfig.fromEnv()` (`utils/security_config.dart`) exige segredo ≥ 32 caracteres e que não seja nenhum dos exemplos públicos.
- Com `APP_ENV=production` a API **recusa subir** com segredo ausente/fraco (falha antes de abrir o banco).
- Em desenvolvimento, segredo vazio/fraco gera um segredo **aleatório temporário** (48 bytes, `Random.secure`) com aviso — o "plug and play" local continua e nenhuma chave conhecida é aceita.
- `.env.example` sem segredo reutilizável, com instrução `openssl rand -base64 48`. Tokens agora levam `iss` e `jti` e a verificação exige o emissor.

```
-  secret: env('JWT_SECRET') ?? 'dev-secret-troque-em-producao',
+  secret: config.jwtSecret,   // forte (>=32) ou aleatório efêmero em dev; produção falha se fraco
```

Verificação: R-01 (3 testes)

```
$ APP_ENV=production JWT_SECRET=dev-secret-troque-em-producao dart run bin/server.dart
Bad state: JWT_SECRET ausente ou fraco (minimo 32 caracteres e diferente dos exemplos).
Gere um segredo forte com: openssl rand -base64 48

teste: token forjado com o segredo antigo  -> HTTP 401
teste: token legitimo                      -> HTTP 200
```

Risco residual: Segredo ainda é um valor de ambiente (ver R-12: cofre de segredos e rotação).

R-02 — IDOR na ingestão de telemetria (POST /sessions/:id/events) Extremo

Categoria: Técnico • **Referência:** A01:2021 – Quebra de Controle de Acesso

Antes: prob. Alta × impacto Alto = Extremo → **Depois (residual):** prob. Muito Baixa × impacto Alto = Baixo • **Status:** Concluído

Descrição: O endpoint de eventos não conferia a posse da sessão: qualquer usuário autenticado injetava telemetria na sessão de outro atleta.

Evidência original: Entrega 1: atacante gravou `ballSpeedKmh: 999` na sessão da vítima (HTTP 201).

Resposta implementada (Prevenir (Evitar)):

- `session_routes.dart`: a rota carrega a sessão e exige `session.user_id == request.userId` (mesmo padrão do `GET /:id/events`); caso contrário **404** (não revela a existência do ID).
- IDs que não são UUID retornam 404 antes de chegar ao banco.

```
+ final session = await db.getSessionById(id);
+ if (session == null || session['user_id'] != request.userId) {
+   return ApiResponse.notFound('Sessao nao encontrada');
+ }
  await db.insertSessionEvents(id, parsed);
```

Verificação: R-02 (1 teste)

```
atacante  POST /sessions/<id-vitima>/events -> HTTP 404, 0 eventos gravados
dono      POST /sessions/<id-vitima>/events -> HTTP 201, 1 evento gravado
```

Risco residual: —

R-03 — IDOR em curtidas e comentários (/feed/:id/like, /comments) Médio

Categoria: Técnico • **Referência:** A01:2021 – Quebra de Controle de Acesso

Antes: prob. Média × impacto Médio = Médio → **Depois (residual):** prob. Muito Baixa × impacto Médio = Baixo • **Status:** Concluído

Descrição: Era possível curtir/comentar (e ler comentários de) qualquer sessão pelo ID, mesmo de usuários não seguidos ou sessões ainda em andamento.

Evidência original: Entrega 1: comentário e curtida aceitos (HTTP 201/200) em sessão de terceiro.

Resposta implementada (Mitigar):

- `feed_routes.dart`: função `canInteract` — a sessão precisa ser do próprio usuário, ou **concluída e de alguém que ele segue** (a mesma regra do feed, então a experiência do app não muda).
- Aplicada em curtir, listar e criar comentários; comentário limitado a 500 caracteres.

```
+ if (ownerId == viewerId) return true;
+ if (session['status'] != 'completed') return false;
+ return db.isFollowing(viewerId, ownerId);
```

Verificação: R-03 (2 testes)

```
estranho  like/comment/GET comments -> HTTP 404 (sessão em andamento e concluída)
seguidor  sessão em andamento -> 404 ; concluída -> like 200 / comment 201
dono      sempre permitido
```

Risco residual: —

R-04 — Enumeração de usuários no cadastro Médio

Categoria: Técnico • **Referência:** A07:2021 – Falhas de Identificação e Autenticação

Antes: prob. Alta × impacto Baixo = Médio → **Depois (residual):** prob. Média × impacto Baixo = Médio • **Status:** Em Andamento

Descrição: O cadastro distingue e-mail já cadastrado (409) de novo (201).

Evidência original: Entrega 1: 409 para e-mail existente, 201 para inexistente.

Resposta implementada (Mitigar + Aceitar (Ativo)):

- Limite de **10 cadastros/hora por IP** (`AuthLimiters.registerPerIp`) → HTTP 429 com `Retry-After`: impede varrer listas de e-mails em escala.
- Login: bcrypt "descartável" quando o e-mail não existe, igualando o tempo de resposta (ver R-15).
- **Decisão consciente:** o 409 foi mantido porque o app o usa para orientar o usuário; trocá-lo exige fluxo de confirmação por e-mail (não existe ainda). Risco residual aceito de forma ativa (rate limit + monitoramento) até a implantação do e-mail de confirmação.

Verificação: R-04/R-05 (cadastro: limite por IP)

```
cadastro #1..#3 -> 201 ; cadastro #4 (mesmo IP, limite=3 no teste) -> 429
```

Risco residual: Enumeração lenta (≤10 e-mails/h/IP) ainda é possível; fecha com confirmação por e-mail.

R-05 — Ausência de proteção contra força bruta (login) Alto

Categoria: Técnico • **Referência:** A07:2021 – Falhas de Identificação e Autenticação

Antes: prob. Alta × impacto Médio = Alto → **Depois (residual):** prob. Baixa × impacto Médio = Baixo • **Status:** Concluído

Descrição: Nenhum limite de tentativas de login (20/20 tentativas processadas na Entrega 1).

Evidência original: Entrega 1: 20 tentativas seguidas, todas 401, nenhuma 429.

Resposta implementada (Mitigar):

- `utils/rate_limiter.dart`: limitador de janela deslizante (15 min) em três camadas — 30 tentativas/IP, **5 falhas por IP+e-mail**, 20 falhas por e-mail (qualquer IP).
- Bloqueio responde **429 + Retry-After**; login correto zera o contador do par. O par IP+e-mail evita que um atacante tranque a conta da vítima a partir de outro IP.
- IP real só é lido de `X-Forwarded-For` quando `TRUST_PROXY=true` (senão o atacante forjaria o header).

Verificação: R-04/R-05 (5 testes)

```
5 falhas (mesmo IP+e-mail) -> HTTP 401 x5
6ª tentativa (senha correta)   -> HTTP 429, Retry-After presente
outro IP, senha correta         -> HTTP 200 (vítima não fica trancada)
```

Risco residual: Estado em memória: por instância da API. Com várias réplicas, mover para Redis/gateway.

R-06 — Política de senha fraca Médio

Categoria: Técnico • **Referência:** A07:2021 – Falhas de Identificação e Autenticação

Antes: prob. Média × impacto Médio = Médio → **Depois (residual):** prob. Baixa × impacto Médio = Baixo • **Status:** Concluído

Descrição: Senhas como `123456` eram aceitas.

Evidência original: Entrega 1: conta criada com senha "123456" (HTTP 201).

Resposta implementada (Mitigar):

- `utils/validators.dart › validatePassword`: 8–128 caracteres, ao menos uma letra e um número, fora de lista de senhas comuns, sem conter o e-mail/ser igual ao nome. O teto de 128 evita DoS no bcrypt.
- App: validador da tela de cadastro alinhado (`signup_screen.dart`).
- **Compatibilidade:** a política vale só no *cadastro*; o login continua aceitando senhas antigas de 6+ caracteres, então nenhuma conta existente é bloqueada.

Verificação: R-06 (3 testes)

```
'123456','12345678','abcdefgh','password1','Ab1', 129 chars -> HTTP 400
'Tenis2026Forte' -> 201 ; conta legada com '123456' ainda faz login (200)
```

Risco residual: Falta checagem contra vazamentos (HaveIBeenPwned k-anonymity) e 2FA — melhorias futuras.

R-07 — CORS permissivo (Access-Control-Allow-Origin: *) Baixo

Categoria: Técnico • **Referência:** A05:2021 – Configuração Incorreta de Segurança

Antes: prob. Média × impacto Baixo = Baixo → **Depois (residual):** prob. Muito Baixa × impacto Baixo = Muito Baixo • **Status:** Concluído

Descrição: Qualquer site podia chamar a API pelo navegador.

Evidência original: Entrega 1: `access-control-allow-origin: *` para `Origin: https://site-malicioso.com`.

Resposta implementada (Mitigar):

- `middleware/security_middleware.dart › corsAllowlist`: allowlist por `CORS_ALLOWED_ORIGINS`; origem permitida é refletida com `Vary: Origin`; preflight de origem não permitida → 403.
- Apps nativos (Android/iOS) não enviam `Origin` e não são afetados. Fora de produção `localhost` segue liberado (`flutter run -d chrome` continua funcionando).

Verificação: R-07 (3 testes)

```
Origin: https://site-malicioso.com -> sem Access-Control-Allow-Origin
Origin allowlist -> refletida ; OPTIONS de origem inválida -> 403
localhost: liberado em dev, bloqueado em produção
```

Risco residual: —

R-08 — Tokens JWT de longa duração sem revogação Médio

Categoria: Técnico • **Referência:** A07:2021 – Falhas de Identificação e Autenticação

Antes: prob. Média × impacto Médio = Médio → **Depois (residual):** prob. Baixa × impacto Médio = Baixo • **Status:** Concluído

Descrição: Tokens de 7 dias, sem logout no servidor nem lista de revogação.

Evidência original: Entrega 1: `exp − iat = 604800 s (168 h)` ; sem endpoint de logout.

Resposta implementada (Mitigar):

- Validade padrão **24 h**, teto de 72 h (`JWT_EXPIRES_IN_HOURS` acima disso é reduzido com aviso).
- Cada token ganha um `jti` ; novo `POST /auth/logout` o inclui numa lista de revogação em memória (limpa após o `exp`). O app chama o logout em modo "melhor esforço" (offline não impede sair).
- Tokens emitidos antes desta versão (sem `jti`) deixam de valer: o app trata o 401 no `bootstrap()` e leva o usuário ao login — **um novo login único** após a atualização.

Verificação: R-08 (1 teste)

```
payload: exp − iat <= 86400 s ; jti presente
logout -> 200 ; mesmo token em /users/me -> 401
```

Risco residual: Revogação em memória se perde ao reiniciar a API (mitigado pela validade curta). Refresh token rotativo = evolução futura.

R-09 — Transporte sem TLS (HTTP em texto claro) Alto

Categoria: Técnico • **Referência:** A02:2021 – Falhas Criptográficas

Antes: prob. Média × impacto Alto = Alto → **Depois (residual):** prob. Baixa × impacto Alto = Médio • **Status:** Em Andamento

Descrição: Tráfego app ↔ API em HTTP puro permite interceptar credenciais e tokens em redes não confiáveis.

Evidência original: Entrega 1: `http://10.0.2.2:8080` / `http://localhost:8080` , sem proxy TLS.

Resposta implementada (Mitigar (+ Transferir TLS ao proxy/provedor)):

- **API:** TLS nativo opcional (`TLS_CERT_PATH` / `TLS_KEY_PATH`) ou atrás de proxy com `TRUST_PROXY=true` ; **HSTS** em produção/HTTPS; aviso na subida se produção rodar em HTTP puro; demais cabeçalhos (`nosniff` , `X-Frame-Options` , `CSP` , `Cache-Control: no-store` , `Referrer-Policy`).
- **Android:** `usesCleartextTraffic="false"` no manifesto principal (release); debug/profile mantêm HTTP para a API local — o desenvolvimento não muda.
- **App:** em build de *release*, `ApiClient` bloqueia URL `http://` que não seja localhost/emulador; basta compilar com `--dart-define=API_BASE_URL=https://…` .
- **Pendente (implantação):** publicar a API com certificado válido (proxy/plataforma de nuvem). O código está pronto; o certificado depende do ambiente de hospedagem.

Verificação: R-09 (2 testes)

```
resposta de produção: strict-transport-security: max-age=31536000; includeSubDomains
x-content-type-options: nosniff · x-frame-options: DENY · cache-control: no-store
```

Risco residual: Enquanto a API não for publicada com HTTPS, o tráfego continua em claro (ambiente acadêmico/local).

R-10 — Container executando como root em modo de desenvolvimento Baixo

Categoria: Organizacional • **Referência:** A05:2021 – Configuração Incorreta de Segurança

Antes: prob. Baixa × impacto Médio = Baixo → **Depois (residual):** prob. Muito Baixa × impacto Médio = Baixo • **Status:** Concluído

Descrição: Imagem rodava `dart run` (JIT) como root.

Evidência original: Entrega 1: `CMD ["dart","run","bin/server.dart"]`, sem `USER`.

Resposta implementada (Mitigar):

- `Dockerfile` multi-estágio: `dart compile exe` (AOT) e imagem final `FROM scratch` (sem shell/gerenciador de pacotes), `USER 10001`, `HEALTHCHECK` pelo próprio binário (`--healthcheck`).
- `docker-compose.yml`: `read_only`, `cap_drop: ALL`, `no-new-privileges`, limites de memória e processos.

Verificação: Compilação AOT e healthcheck testados localmente; build Docker não executado (Docker indisponível no ambiente).

```
FROM scratch ... USER 10001:10001
HEALTHCHECK CMD ["/app/bin/server","--healthcheck"]
(dart compile exe: OK ; --healthcheck sem servidor: exit 1)
```

Risco residual: Validar `docker compose up --build` na máquina do grupo.

R-11 — Falta de validação/limite de payload (abuso de armazenamento/DoS) Baixo

Categoria: Técnico • **Referência:** A04:2021 – Design Inseguro

Antes: prob. Média × impacto Baixo = Baixo → **Depois (residual):** prob. Baixa × impacto Baixo = Baixo • **Status:** Concluído

Descrição: Listas de eventos e textos sem limite.

Evidência original: Entrega 1: `events` de tamanho arbitrário; comentários/notas sem limite.

Resposta implementada (Mitigar):

- `bodyLimit`: corpo máximo de 256 KiB (`MAX_BODY_BYTES`), checado por `Content-Length` e em streaming (cobre *chunked*) → 413.
- Máx. 500 eventos por requisição (o app envia lotes pequenos a cada 6 s); `eventType` restrito a `[a-z][a-z0-9_]{0,39}` (aceita `shot/ace/fault` e os tipos futuros do ESP32); faixas: velocidade 0–400 km/h, rotação 0–10000 rpm, ângulo ±360°.
- Limites de texto: comentário 500, notas 2000, bio 300, título 120 etc.; números das sessões/metas com faixa (evita *overflow* de `NUMERIC(5,1)` → 500).

Verificação: R-11 (3 testes)

```
501 eventos -> 413 (0 gravados) ; corpo de 300 KiB -> 413
speed=99999 / 'rapida' / eventType com SQL / evento não-objeto -> 400
```

Risco residual: Sem cota global de eventos por sessão/usuário (baixo risco).

R-12 — Exposição de credenciais do banco (Supabase) por vazamento de segredo Alto

Categoria: Externo • **Referência:** A02/A05 – Gestão de Segredos

Antes: prob. Baixa × impacto Muito Alto = Alto → **Depois (residual):** prob. Baixa × impacto Muito Alto = Médio • **Status:** Em Andamento

Descrição: Um `.env` commitado, logado ou copiado para uma imagem expõe o PostgreSQL do Supabase.

Evidência original: Entrega 1: `.env` no `.gitignore`, mas sem cofre, sem verificação em CI e sem `.dockerignore` (o `COPY . .` do Dockerfile levaria o `.env` para a imagem).

Resposta implementada (Transferir + Mitigar):

- `.dockerignore` exclui `.env*` da imagem (o `.env` chega só como variável de ambiente via compose).
- Workflow `.github/workflows/security.yml`: **gitleaks** em cada push/PR + `dart analyze` e `dart test`.
- Auditoria do histórico git: nenhum `.env` real ou senha foi versionado (só os placeholders do `.env.example`).
- **Transferir:** usar Supabase Vault / GitHub Actions Secrets para produção. **Pendente:** rotação periódica e papel de banco com menor privilégio.

Verificação: Auditoria manual do histórico; workflow só roda no GitHub.

```
git log --all -p: DB_PASSWORD=<senha-do-banco> / troque-pela-senha-do-banco (apenas placeholders)
```

Risco residual: Rotação de senha e usuário de banco restrito ainda por fazer.

R-13 — Exposição de dados pessoais no perfil público (GET /users/:id) Alto novo nesta entrega

Categoria: Técnico • **Referência:** A01:2021 – Quebra de Controle de Acesso / A02 – Exposição de Dados

Antes: prob. Alta × impacto Médio = Alto → **Depois (residual):** prob. Muito Baixa × impacto Médio = Baixo • **Status:** Concluído

Descrição: Encontrado nesta entrega por revisão de código: `GET /users/:id` devolvia `userToJson` completo para qualquer usuário autenticado — **e-mail, data de nascimento e role** de terceiros.

Evidência original: SAST: `user_routes.dart › fetchUserWithStats` reutilizava o mapper do dono para qualquer `id`. O app só usa e-mail do próprio usuário (`AppUser.email` é opcional).

Resposta implementada (Prevenir (Evitar)):

- Novo mapper `userPublicToJson` (`mappers.dart`): nome, avatar, bio, nível, cidade e data de cadastro. E-mail/nascimento/role só voltam em `/users/me` e `PUT /users/me`.

Verificação: Privacidade do perfil público (1 teste)

```
GET /users/<outro>  -> sem email, birthDate, role
GET /users/me       -> email presente
```

Risco residual: —

R-14 — Sem tratamento global de erros (HTTP 500 / vazamento) e IDs sem validação Médio novo nesta entrega

Categoria: Técnico • **Referência:** A05:2021 – Configuração Incorreta / A09 – Falhas de Log e Monitoramento

Antes: prob. Alta × impacto Baixo = Médio → **Depois (residual):** prob. Baixa × impacto Baixo = Baixo • **Status:** Concluído

Descrição: Encontrado por revisão de código: não havia `errorHandler`. JSON malformado, `as Map` / `as String` sobre tipos errados e UUIDs inválidos (`@id::uuid` no Postgres) terminavam em exceção não tratada → 500, expondo comportamento interno e facilitando sondagem.

Evidência original: SAST: `readJsonBody` lança `FormatException`; `event as Map<String,dynamic>` lança `TypeError`; `GET /users/abc` → erro 22P02 do driver.

Resposta implementada (Mitigar):

- `errorHandler`: JSON inválido/tipos errados → 400; erros de dado do Postgres (22P02, 22003, 22001, 23503…) → 400 "Dados invalidos"; qualquer outro → 500 genérico "Erro interno" (detalhes só no log do servidor).
- IDs de rota validados com `isUuid` antes de consultar o banco (404 para formato inválido). Resposta de erro nunca inclui stack trace/SQL.

Verificação: R-11 (validação) + "erros internos nao vazam detalhes"

```
'{nao-e-json' -> 400 ; '[1,2]' -> 400 ; evento 'nao-e-objeto' -> 400 ; /sessions/nao-uuid -> 404
exceção interna -> 500 {"error":"Erro interno"} (sem 'Unimplemented...' no corpo)
```

Risco residual: —

R-15 — Diferença de tempo no login revela se o e-mail existe (timing) Médio novo nesta entrega

Categoria: Técnico • **Referência:** A07:2021 – Falhas de Identificação e Autenticação

Antes: prob. Média × impacto Baixo = Médio → **Depois (residual):** prob. Baixa × impacto Baixo = Baixo • **Status:** Concluído

Descrição: Quando o e-mail não existia, o login respondia sem executar o bcrypt (~100 ms mais rápido), permitindo enumerar contas pelo tempo de resposta mesmo com a mensagem genérica.

Evidência original: SAST: `auth_routes.dart` retornava 401 antes de `PasswordUtil.verify` quando `findUserByEmail` era nulo.

Resposta implementada (Mitigar):

- Um hash bcrypt descartável é verificado também quando o usuário não existe, equalizando o custo.

Verificação: Coberto indiretamente pelos testes de login; sem teste de tempo dedicado (medição de tempo é instável em CI).

```
usuário inexistente -> bcrypt executado (mesmo custo) -> 401 'Email ou senha incorretos'
```

Risco residual: —

R-16 — Validação insuficiente de entrada e integridade dos dados Médio novo nesta entrega

Categoria: Técnico • **Referência:** A04:2021 – Design Inseguro / A03 – Injeção (dados)

Antes: prob. Média × impacto Médio = Médio → **Depois (residual):** prob. Baixa × impacto Baixo = Baixo • **Status:** Concluído

Descrição: Rotas gravavam valores sem checar domínio: `avatarUrl` com qualquer esquema (`javascript:`, `data:`), `level`/`status` arbitrários, estatísticas negativas ou absurdas (*overflow* → 500), `deviceId` de **outro usuário** ao criar sessão, seguir usuário inexistente (erro de FK) e nome/bio sem limite.

Evidência original: SAST em `user_routes`, `session_routes`, `goal_routes`, `device_routes`, `training_routes`.

Resposta implementada (Mitigar):

- Validadores centrais (`utils/validators.dart`): `validateText/Number/HttpUrl/Date`, allowlists de `level` e `status` da meta, cor hexadecimal, `isSlug` para tipos.
- `POST /sessions`: `trainingId` deve existir e `deviceId` deve pertencer ao usuário autenticado. `POST /users/:id/follow` valida existência do alvo.
- **Compatibilidade:** os valores que o app envia hoje (`iniciante…profissional`, `active/completed`, tipos de meta e eventos `shot/ace/fault`) foram conferidos no código do app e continuam aceitos.

Verificação: R-11 (validação de faixas/tipos)

```
avatarUrl 'javascript:alert(1)' -> 400 ; level 'root' -> 400 ; shotCount -5 -> 400 ; deviceId de terceiro -> 404
```

Risco residual: —

R-17 — Curingas de ILIKE sem escape na busca de usuários Baixo novo nesta entrega

Categoria: Técnico • **Referência:** A03:2021 – Injeção (padrão de busca)

Antes: prob. Média × impacto Muito Baixo = Baixo → **Depois (residual):** prob. Muito Baixa × impacto Muito Baixo = Muito Baixo • **Status:** Concluído

Descrição: `GET /users/search?q=%` casava todos os usuários (e `_` qualquer caractere), facilitando varrer a base. Não é injeção de SQL (a consulta é parametrizada).

Evidência original: SAST: `pattern: '%$query%'` em `postgres_store.searchUsers`.

Resposta implementada (Prevenir (Evitar)):

- `escapeLike` escapa `% _ \` e a consulta usa `ILIKE … ESCAPE '\'`; consulta limitada a 50 caracteres (e 20 resultados, já existente).

Verificação: Alteração em SQL — verificar contra o PostgreSQL real (ver Seção 8).

```
q='%' agora procura o caractere literal '%'
```

Risco residual: —

R-18 — Token JWT guardado sem criptografia no dispositivo (SharedPreferences) Médio novo nesta entrega

Categoria: Técnico • **Referência:** A02:2021 – Falhas Criptográficas (M9 – Armazenamento Inseguro, OWASP Mobile)

Antes: prob. Baixa × impacto Alto = Médio → **Depois (residual):** prob. Baixa × impacto Alto = Médio • **Status:** Não Iniciado

Descrição: `ApiClient` grava o token em `SharedPreferences` (texto claro, incluído em backups e legível em dispositivo com root/jailbreak; *localStorage* na web).

Evidência original: SAST: `api_client.dart › saveToken`.

Resposta implementada (Mitigar (planejado)):

- Recomendado: migrar para `flutter_secure_storage` (Keystore/Keychain). **Não aplicado nesta entrega** porque exige nova dependência, ajuste nativo (minSdk 23, Keychain) e testes em dispositivo que não foi possível executar aqui; trocar às cegas arriscaria quebrar o login do app.
- Mitigação parcial já ativa: validade de 24 h e revogação no logout (R-08).

Verificação: —

```
–
```

Risco residual: Aceito até a migração; priorizar antes de publicar o app em loja.

6. Verificação: Suíte de Testes de Segurança

Arquivo: `SRC/Swing Sense/swing_sense_backend/test/security_test.dart` (executar com `dart test`). Os testes montam o pipeline completo da API (`buildHandler`: CORS, cabeçalhos, limite de corpo, tratamento de erro, autenticação e rotas) sobre um banco **em memória** e reproduzem os ataques do relatório da Entrega 1. Resultado desta execução: **25 testes, todos aprovados**; `dart analyze` sem avisos.

```
$ dart test test/security_test.dart
00:07 +25: All tests passed!
```

1. Privacidade do perfil publico perfil de terceiros nao expoe email, nascimento nem role
2. R-01 segredo JWT segredos padrao/fracos sao reconhecidos
3. R-01 segredo JWT token forjado com o segredo antigo do repositorio e rejeitado
4. R-01 segredo JWT token sem jti/issuer (assinado com a chave certa) tambem e rejeitado
5. R-02 IDOR na telemetria atacante nao grava eventos na sessao da vitima; dono grava
6. R-03 IDOR em curtidas/comentarios comentario acima de 500 caracteres e recusado
7. R-03 IDOR em curtidas/comentarios so dono ou seguidor (sessao concluida) interage
8. R-04/R-05 forca bruta e abuso cadastro: limite por IP
9. R-04/R-05 forca bruta e abuso injecao de SQL no login continua falhando de forma segura
0. R-04/R-05 forca bruta e abuso janela deslizante libera apos o prazo
1. R-04/R-05 forca bruta e abuso login bem sucedido zera o contador de falhas
2. R-04/R-05 forca bruta e abuso login: 5 falhas do mesmo IP+email => 429 com Retry-After
3. R-06 politica de senha senha forte e aceita e login antigo (6+ caracteres) segue possivel
4. R-06 politica de senha senhas fracas sao recusadas no cadastro
5. R-06 politica de senha validatePassword
6. R-07 CORS em desenvolvimento localhost continua liberado (flutter run -d chrome)
7. R-07 CORS origem da allowlist e refletida; preflight de origem invalida = 403
8. R-07 CORS origem desconhecida nao recebe Access-Control-Allow-Origin
9. R-08 vida e revogacao do token exp - iat <= 24h e logout invalida o token
0. R-09 cabecalhos / TLS HSTS em producao
1. R-09 cabecalhos / TLS cabecalhos de seguranca presentes
2. R-11 limites e validacao corpo gigante => 413 (Content-Length e streaming)
3. R-11 limites e validacao mais de 500 eventos => 413
4. R-11 limites e validacao valores fora de faixa / tipos errados / JSON invalido => 400 (nunca 500)
5. erros internos nao vazam detalhes

Cada item acima é um grupo + caso de teste.

Outras verificações executadas

- **Subida com configuração insegura:** `APP_ENV=production` + segredo padrão → a API aborta antes de conectar ao banco (R-01). Em desenvolvimento sem segredo → aviso e segredo temporário.
- **Compilação AOT** (`dart compile exe bin/server.dart`) concluída — base do novo Dockerfile (R-10); `--healthcheck` retorna código 1 quando não há servidor.
- **App Flutter:** alterações em `signup_screen.dart`, `auth_service.dart`, `api_client.dart` e manifestos Android revisadas e analisadas sintaticamente.

7. Mapa das Alterações no Código

Arquivo (em `SRC/Swing Sense/`)	Alteração	Riscos

`swing_sense_backend/lib/src/utils/security_config.dart` *(novo)*	Configuração de segurança: segredo JWT forte/efêmero, validade, CORS, proxy, limite de corpo	R-01, R-07, R-08
`…/middleware/security_middleware.dart` *(novo)*	Cabeçalhos de segurança/HSTS, CORS por allowlist, limite de corpo, tratamento de erros, IP do cliente	R-07, R-09, R-11, R-14
`…/utils/rate_limiter.dart` *(novo)*	Limitador de janela deslizante	R-04, R-05
`…/utils/validators.dart` *(novo)*	Validação de UUID, senha, textos, números, URL, datas; escape de LIKE	R-06, R-11, R-16, R-17
`…/utils/jwt_util.dart`	`jti`, verificação de emissor e claims, revogação	R-01, R-08
`…/routes/auth_routes.dart`	Rate limit, política de senha, anti-timing, `/auth/logout`	R-04, R-05, R-06, R-08, R-15
`…/routes/session_routes.dart`	Posse na telemetria, validação de eventos/estatísticas, device/training do usuário	R-02, R-11, R-16
`…/routes/feed_routes.dart`	Visibilidade para curtir/comentar, limite de texto	R-03, R-11
`…/routes/user_routes.dart`, `mappers.dart`, `db/postgres_store.dart`	Perfil público sem PII, validação de perfil, busca com escape, follow validado	R-13, R-16, R-17
`…/routes/goal_routes.dart`, `device_routes.dart`, `training_routes.dart`	Validação de entrada e UUIDs	R-11, R-16
`…/app.dart`, `bin/server.dart`	Novo pipeline de middlewares; TLS nativo opcional; `--healthcheck`	R-07, R-09, R-10, R-14
`…/Dockerfile`, `docker-compose.yml`, `.dockerignore`, `.env.example`, `README.md`	Imagem AOT não-root, hardening, exclusão de segredos, variáveis documentadas	R-01, R-10, R-12
`…/test/security_test.dart` *(novo)*	25 testes de segurança	todos
`swing_sense_frontend/lib/screens/auth/signup_screen.dart`	Validação de senha igual à da API	R-06
`…/lib/services/auth_service.dart`, `api_client.dart`	Logout com revogação; bloqueio de HTTP em release	R-08, R-09
`…/android/app/src/{main,debug,profile}/AndroidManifest.xml`	Cleartext proibido em release, permitido em debug/profile	R-09
`.github/workflows/security.yml` *(novo, raiz do repositório)*	gitleaks + análise + testes em CI	R-12

8. Limitações da Verificação e Pontos de Atenção

Registro transparente do que **não** pôde ser comprovado no ambiente de elaboração:

- **Banco real:** não havia PostgreSQL/Supabase disponível; os testes usam banco em memória. A única alteração de SQL é a cláusula `ESCAPE '\'` na busca de usuários (R-17) — validar com `GET /users/search?q=ana` no Supabase.
- **Docker:** o build da nova imagem não foi executado (Docker indisponível). Rodar `docker compose up -d --build` e conferir `docker ps` (estado *healthy*).
- **App Flutter:** o Flutter local está desatualizado e não resolve as dependências (`google_fonts` exige Dart ≥ 3.9); não foi possível rodar `flutter analyze` /o app. Recomenda-se um teste manual: cadastro (senha com letra+número), login, iniciar/encerrar sessão, curtir/comentar no feed, editar perfil, sair e entrar de novo.
- **Dependência pré-existente:** o `pubspec.yaml` do backend pede `postgres ^3.5.12`, que exige Dart ≥ 3.9. Os testes foram executados em Dart 3.5.4 numa cópia temporária com `postgres 3.5.7` (o mesmo da Entrega 1); o `pubspec.yaml` do projeto não foi alterado.
- **Riscos que dependem de implantação:** HTTPS real (R-09), cofre/rotação de segredos (R-12) e confirmação por e-mail (R-04).

9. Monitoramento e Revisão

Este plano será revisado **a cada sprint (quinzenalmente)** nas reuniões de status e a cada marco de entrega. O campo "Status" é atualizado conforme o progresso (Não Iniciado, Em Andamento, Concluído, Cancelado). Todo risco *Concluído* tem teste automatizado que roda em CI a cada push (workflow `security.yml`), de modo que uma regressão reabre o risco automaticamente. Novos riscos são adicionados ao registro quando surgirem — em especial na integração com o ESP32 (autenticação do dispositivo, BLE e proteção da telemetria), que ainda não foi coberta.

Próximos passos priorizados: (1) publicar a API com HTTPS e definir `APP_ENV=production`; (2) migrar o token do app para armazenamento seguro (R-18); (3) confirmação de e-mail e fechamento do R-04; (4) rotação de segredos e usuário de banco com menor privilégio (R-12); (5) rate limit distribuído se houver mais de uma réplica.

Aprovação

Nome	Cargo	Assinatura	Data
Lucas Silva Maciel	Gerente do Projeto / Coord. de Segurança	*Lucas S. Maciel*	19/10/2026
Antônio Petri	Equipe do Projeto	*Antônio Petri*	19/10/2026
Gabriel Higobassi Paschoal	Equipe do Projeto	*Gabriel H. Paschoal*	19/10/2026
Thiago Akira Higa Mitami	Equipe do Projeto	*Thiago A. H. Mitami*	19/10/2026

Anexo — Metodologia: revisão manual de código (SAST) de rotas, middlewares, utilitários, schema, Dockerfile e app; testes automatizados que reproduzem os ataques (DAST sobre o pipeline HTTP em memória); mapeamento ao OWASP Top 10 (2021) e OWASP Mobile Top 10. Documento-base de estrutura: modelo "Plano de Respostas aos Riscos".